AdaConfigure: Reinforcement Learning-based Adaptive Configuration for Video Analytics Services

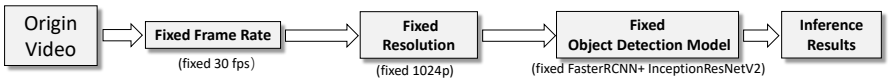
Zhaoliang He[1,4], Yuan Wang[2], Chen Tang[3], Zhi Wang*[3], Wenwu Zhu[1], Chenyang Guo[5], and Zhibo Chen[5]

[1] Department of Computer Science and Technology, Tsinghua University
hezl19@mails.tsinghua.edu.cn,wwzhu@tsinghua.edu.cn
[2] Tsinghua-Berkeley Shenzhen Institute, Tsinghua University
wangyuan19@mails.tsinghua.edu.cn
[3] Tsinghua Shenzhen International Graduate School, Tsinghua University
tc20@mails.tsinghua.edu.cn,wangzhi@sz.tsinghua.edu.cn
[4] Peng Cheng Laboratory
[5] Tencent Youtu Lab
{chenyangguo,ruibobchen}@tencent.com

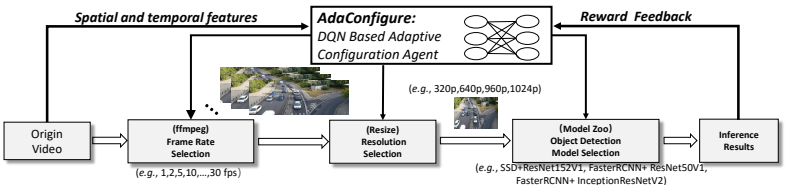
Abstract. The configuration in video analytics defines parameters including frame rate, image resolution, and model selection for video analytics pipeline, and thus determines the inference accuracy and resource consumption. Traditional solutions to select a configuration are either fixed (i.e., the same configuration is used all the time) or periodically adjusted using a brute-force search scheme (i.e., periodically trying different configurations and selecting the one with the best performance), and thus suffer either low inference accuracy or high computation cost to find a proper configuration timely. To this end, we propose a video analytical configuration adaptation framework called AdaConfigure that dynamically selects video configuration without resource-consuming exploration. First, we design a reinforcement learning-based framework in which an agent adaptively chooses the configuration according to the spatial and temporal features of the current video stream. In particular, we use a video segmentation strategy to capture the characteristics of the video stream with much-reduced computation cost: profiling uses only 0.2-2% computation resources as compared to a full video. Second, we design a reward function that considers both the inference accuracy and computation resource consumption so that the configuration achieves good accuracy and resource consumption trade-off. Our evaluation experiments on an object detection task show that our approach outperforms the baseline: it achieves 10-35% higher accuracy with a similar amount of computation resources or achieves similar accuracy with only 10-50% of the computation resources.

Keywords: Adaptive Configuration, Reinforcement Learning, Video Analytics Services

* Zhi Wang is the corresponding author.



(a) Static configuration solution: fixed configuration for video analytics services.



(b) AdaConfigure solution: reinforcement learning-based adaptive configuration framework.

Fig. 1. Comparing to the static solution, our solution can adaptively update the configuration strategy based on the reward feedback.

1 Introduction

With the increasing demand for continuous video analytics in public safety and transportation, more and more cameras are being deployed to various locations. The video analytics are based on classical computer vision techniques as well as deep convolutional neural networks. A video analytics application consists of a *pipeline* of several video processing modules, typically including a decoder, a selective sampling frame application, and a target detector. Such a pipeline always has multiple *knobs*, such as frame rate, resolution, detector model selection and so on. A combination of the knob values is video analytics *configuration*, and the configuration space grows *exponentially* with the number of knobs and their values [10].

Different configurations directly affect accuracy and resource consumption. The best configuration is the one with the lowest resource demand whose accuracy is over the desired threshold, which can optimize the *trade-off* between accuracy and energy consumption. The *best* configuration for video analysis services often varies in minutes or even seconds [10]. As shown in Figure 1(a), if one uses an expensive static configuration (e.g., only profiles the processing pipeline *once* to choose the high frame rate and image resolution) can be precise, but it can also be a huge drain on resources. Similarly, specifying a cheap static configuration (e.g., low resolution and small model) will significantly reduce accuracy. The framework of our solution is shown in Figure 1(b), and we tackle the following design challenges.

Choosing the best configuration is a complicated decision-making problem that is challenging to be solved by rules. The best configuration for video analytics services changes significantly because the exact context of the videos varies over time and across space. For instance, tracking vehicles when traffic moves quickly requires a higher frame rate than when traffic moves slowly. Spatially, the characteristics of video content are different in different locations. For instance, cameras

in downtown areas show more cars than the other cameras deployed in the suburbs. It is hard to make an exact rule to choose a configuration for the current context by profiling such complicated video characteristics.

Adaptive configuration would cause a huge extra overhead. The number of possible configurations grows exponentially, and thousands of configurations can be combined with just a few knobs [10]. So exhaustive periodically (e.g., profile once per 4 seconds) configuration to find the best configuration is a highly unrealistic approach because it causes a huge extra overhead, which may exceed the benefits of adaptive configurations. To significantly reduce the resource cost for adaptive configuration requires one solution that automatically chooses a configuration instead of manually trying to use various configurations, which is challenging to previous studies including [10, 20], since their approaches pay attention to reduce search space algorithm, and try various configurations in the search space to find the best configuration.

In our solution, AdaConfigure can adaptively and automatically select the best configuration according to intrusive dynamics of video context, thus solving this difficult optimal configuration decision problem in a low-cost way. To the best of our knowledge, we are the first to propose an adaptive video configuration solution to automatically choose a configuration instead of manually trying to use various configurations. The main contributions of this paper are summarized as follows.

We propose a Deep Q-learning Network-based [14] agent to adaptively pick the best video analytics configuration according to the characteristics of the video stream. For the agent's state, we extract the spatial and temporal features of the video context, so that the agent can adaptively update configuration over time and achieve a superior performance in the multi-camera situation.

We leverage agent's automatic selection characteristic and the video segmentation strategy to reduce profiling cost. In particular, we divide the video into T-second intervals as video chunks and use the agent to choose the best configuration for the first t seconds of the video chunk. It then sticks with the chosen configuration for the rest of the video chunk ($T - t$ seconds) to reduce profiling number. In the evaluation, the profiling cost is about 0.2-2% of the overall video analytics resource consumption.

We design a reward function that considers both inference accuracy and computation resources to assess each configuration's impact. Also, to meet different accuracy-demand services, we leverage the balance factor in the reward function to train different agents. Our evaluation experiments on an object detection task show that our approach outperforms the baseline: it achieves 10-35% higher accuracy with a similar amount of computation resources or achieves similar accuracy with only 10-50% of the computation resources.

2 Related Works

2.1 Static Configuration Optimization

Several previous papers have considered optimizing video analytics services by either adjusting the configuration knobs or training specialized NN models. VideoStorm [21] profiles thousands of video analytics queries on live video streams over large clusters, achieving resource-quality tradeoff with multi-dimensional configurations. VideoEdge [9] introduces *dominant demand* to identify the best tradeoff between multiple resources and accuracy. MCDNN [4] provides a heuristic scheduling algorithm to select model variants for deep stream processing under resource constraints. Focus [7] deconstructs video analytics into two phases (i.e., video ingests and video query), achieving an effective and flexible tradeoff of video analytics's latency and accuracy. These algorithms all profile and optimize video analytics only once at the beginning of the video. In their works, video is divided into pictures at a constant frame rate, and the work of video analysis is regarded as a fixed task either. In other words, they do not handle changes in video stream context. But the optimal configurations do change over time because of the complex and changing video stream context.

2.2 Dynamic Configuration Optimization

Some classic works study dynamic optimization problems. INFaaS [17] automatically selects a model, hardware architecture, and any compiler optimizations and makes scaling and resource allocation decisions. QuickAdapt [19] uses fuzzy rules to enable a Big Data Cyber Security Analytics (BDCA) system to adapt to the changes in security events data quickly. These algorithms do not apply to video configuration.

Some papers study how to dynamically optimize the configuration for video analytics when the video stream context changes. JCAB [20] jointly optimizes configuration adaption, and bandwidth allocation to address several critical challenges in edge-based video analytics systems, including edge capacity limitation, unknown network variation, intrusive dynamics of video context. Chameleon [10] leverages temporal and spatial correlation to decrease the cost of profiling, and exploits the knob independence to reduce the search space from exponential to linear.

Notice these reduce search space algorithms still trying various configurations to find the best configuration. Using different configurations lead to extra expensive profiling costs. To significantly reduce the cost of profiling, we propose an automatic and adaptive configuration algorithm. In particular, we leverage agent's automatic selection characteristic instead of manually trying to use various configurations, achieving an enormous reduction in profiling cost.

3 Reinforcement Learning-based Adaptive Configuration

Figure 2 summarizes how Reinforcement Learning (RL) can be applied to the adaptive configuration. Briefly, it is a reinforcement learning-based system to

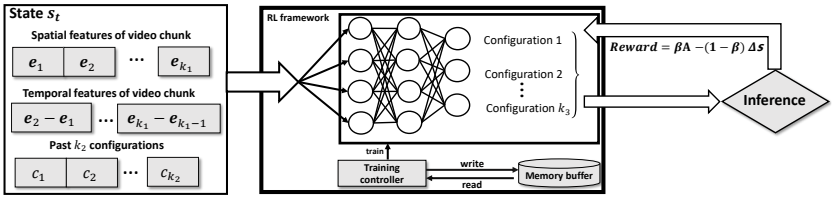


Fig. 2. Applying reinforcement learning to adaptive configuration.

train an agent to choose a proper configuration c for one video chunk for inference. We discuss the formulation, agent design, reinforcement learning-based framework, reward feedback in the following subsections. We provide experimental details of all the hyperparameters in Section 4.

3.1 Problem Formulation

To adaptively choose different configurations for a video stream, we divide the video into T-second intervals as video chunks, and profiles configurations for each video chunk. Without loss of generality, we denote the object detection service as $\boldsymbol{y}_i = M(x_i)$ that provides a predicted result list $\boldsymbol{y}_i$ for each input video chunk x_i. It has a baseline output $\boldsymbol{y}_{\text{ref}} = M(x_{\text{ref}})$ for each input video chunk $x \in X_{\text{ref}}$ using *reference configuration* (the most expensive configuration). We use this $\boldsymbol{y}_{\text{ref}}$ as the ground truth label. For each video chunk x_c that uses a configuration c, the output is $\boldsymbol{y}_c = M(x_c)$. Therefore, we have an accuracy metric $\mathcal{A}_c$ by comparing $\boldsymbol{y}_{\text{ref}}$ and $\boldsymbol{y}_c$. In general, we use the F1 score as the accuracy $\mathcal{A}$ and average GPU processing time per frame as the metric of resource consumption, detailed described in Section 4.2. We also denote the metric of resource consumption as $\hat{s}_{ic}$ that for an input video chunk x_i and a given configuration c. For a reference configuration c_{ref}, the reference resource consumption is $\hat{s}_{\text{ref}}$.

Initially, the agent tries different configurations c to obtain inference results $\boldsymbol{y}_c$ from input video chunk x. To obtain object detection results $\{\boldsymbol{y}_{\text{ref}}, \boldsymbol{y}_c\}$, the agent uses the choosen configuration c and the reference configuration x_{ref}. Comparing the two object detection results $\{\boldsymbol{y}_{\text{ref}}, \boldsymbol{y}_c\}$ and two resource consumptions $\{\hat{s}_{\text{ref}}, \hat{s}_c\}$, the agent computes the resource consumption ratio $\Delta s = \frac{\hat{s}_c}{\hat{s}_{\text{ref}}}$ and accuracy metric $\mathcal{A}_c$.

3.2 RL Agent Design and Video Characteristic Extraction

The RL agent is expected to give a proper configuration c for minimizing the resource consumption $\hat{s}_c$ while keeping the accuracy $\mathcal{A}$. For the RL agent, the input features are continuous numerical vectors, and the expected output is discrete video configuration c. Therefore we can use the Deep Q-learning Network [14] as the RL agent. But the naive Deep Q-learning Network can not work well in this task because the state space of reinforcement learning is too large if we directly treat video chunks as the input, making the RL agent extremely difficult to converge.

To address this challenges, we leverage frame loss strategies of FFmpeg [2] to extract the top k_1 representative images from each chunk and use a pre-trained small neural network to extract the structural characteristics embeddings $\{e_1, e_2, ..., e_{k_1}\}$ of the images as spatial features to reduce the input dimension and accelerate the training procedure. This is a commonly used strategy in training a deep neural network [3, 15]. In this work, we use the early convolution layers of MobileNetV2 [6] as the image feature extractor $\mathcal{E}(\cdot)$ for its efficiency in image recognition. To extract the temporal features of the video chunk, we obtain $\{\hat{e}_1, \hat{e}_2, ..., \hat{e}_{k_1-1}\}$ by each embedding subtracting previous embedding. Besides, we record the last k_2 configurations $\{c_1, c_2, ..., c_{k_2}\}$. To solve that vectors of different lengths are not conducive to input to the neural network, we use the fully connected layer to transform the spatial embedding $\{e_1, e_2, ..., e_{k_1}\}$, temporal embedding $\{\hat{e}_1, \hat{e}_2, ..., \hat{e}_{k_1-1}\}$, and recent configuration $\{c_1, c_2, ..., c_{k_2}\}$ to the fixed length vector, similar to the work [13]. We formulate the fixed length vector s as *states* and the configuration c as discrete *actions*.

We update the RL agent's policy by changing the parameters of the Deep Q-learning Network ϕ while fixing the feature extractor $\mathcal{E}$.

3.3 Reinforcement Learning-based Framework

In our system, we define k_3 discrete actions to indicate k_3 configuration, and the specific configurations are provided in Section 4.1. We denote the *action-value function* as $Q(s, c; \theta)$ and the optimal compression quality level at time t as $c_t = \text{argmax}_c Q(s, c; \theta)$ where θ indicates the parameters of the Deep Q-learning Network ϕ. In such reinforcement learning formulation, the training phase is to minimize the loss function $L_i(\theta_i) = \mathbb{E}_{s,c\sim\rho(\cdot)}\Big[\big(y_i - Q(s, c; \theta_i)\big)^2\Big]$ that changes at each iteration i where target $y_i = \mathbb{E}_{s'\sim\{\mathcal{X},M\}}\big[r + \gamma \max_{c'} Q(s', c'; \theta_{i-1}) \mid s, c\big]$. Especially, r is the reward feedback, and $\rho(s, c)$ is a probability distribution over state s and the configuration c [14]. When minimizing the distance of *action-value function*'s output $Q(\cdot)$ and target y_i, the *action-value function* $Q(\cdot)$ outputs a more accurate estimation of an action.

In the training phase, the RL agent firstly uses a ϵ-greedy method to take some random trials to observe the environment's reaction and decreases the randomness when training afterward. In iteration t, we input state s_t to neural network ϕ. The RL agent ϕ generates a specific configuration c_t. The framework processes the video chunk x_t using configuration c_t to infer object detection services and obtains reward r_t. Then the framework obtains the next video chunk x_{t+1} and generates the next state s_{t+1}. The framework stores transition (s_t, c_t, r_t, s_{t+1}) in a memory buffer $\mathcal{D}$. All transitions are saved into a memory buffer $\mathcal{D}$, and the agent learns to optimize its *action* by minimizing the loss function L on a mini-batch from $\mathcal{D}$. The training procedure would converge when the agent's randomness keeps decaying. Finally, the agent's actions are based on its historical "optimal" experiences. The training procedure is presented in Algorithm 1.

Algorithm 1 Training RL agent ϕ

Initialize action-value function Q with random weights θ, replay memory buffer $\mathcal{D}$ and state s_1
for $t \in 1, 2, \cdots, K$ **do**
  1) Exploration
  With probability ϵ:
    $c_t \leftarrow$ a random valid value
  Otherwise:
    $c_t \leftarrow \text{argmax}_c Q(s_t, c; \theta)$
  2) Reward calcuation
  Process video chunk x_t using configuration c_t for inference
  Obtain $(\boldsymbol{y}_{\text{ref}}, \boldsymbol{y}_c)$ from the object detection service
  Compute reward $r \leftarrow R(\Delta s, \mathcal{A}_c)$ according to 3.4 Reward Feedback
  3) Gradient descent
  Obtain next video chunk x_{t+1}
  Generate next state s_{t+1}
  $\mathcal{D} \leftarrow \mathcal{D} \bigcup \{(s_t, c_t, r_t, s_{t+1}\}$
  Sample a randomly mini-batch of transitions (s_j, c_j, r_j, s_{j+1}) from memory buffer $\mathcal{D}$
  $y_j \leftarrow r_j + \gamma \max_{c'} Q(s_{j+1}, c'; \theta)$
  Perform a gradient descent step on $\Big(y_j - Q(s_j, c_j; \theta)\Big)^2$ according to [14]
end for

3.4 Reward Feedback

In our solution, the agent is trained by the reward feedback. In the above formulation, we define resource consumption rate $\Delta s = \frac{\hat{s}_c}{\hat{s}_{\text{ref}}}$ and accuracy metric $\mathcal{A}_c$ at configuration c. Basically, we want the agent to choose a proper configuration for minimizing the resource consumption while remaining acceptable accuracy. Therefore the overall reward r should be positively correlated with the accuracy $\mathcal{A}$ while negatively with the resource consumption ratio Δs. We introduce a balance factor β to form a linear combination $r = \beta\mathcal{A} - (1 - \beta)\Delta s$ as the *reward function* $R(\Delta s, \mathcal{A})$ so that the configuration achieves good accuracy and resource consumption trade-off.

4 Evaluation

4.1 Experiment Setup

We carry out real-world experiments on the object detection task (basic computer vision services) to verify our solution's performance. We use a desktop PC (Intel Xeon E5-2650 v4 CPU) with four NVIDIA 1080ti graphic card as the server infrastructure, and simulate the environment [8] with three pretrained object detection models in Tensorflow, which are SSD[12]+ResNet152V1[5], FasterRCNN[16]+ResNet50V1, FasterRCNN+InceptionResNetV2[18]. For each model, two kinds of image resolutions, which is 1024×1024 and 640×640, can be chosen. In our experiment, we set the choices space of fps as {1, 2, 5, 10, 15, 20, 25, 30}. We use FFmpeg to switch the frame rate and image resolution, which

decide which frames and in what size they should be fed to the object detection model. The configuration space comprises all possible combinations of values of these knobs, so the environment has 48 configurations in total.

4.2 Datasets and Metric

Most public datasets cannot fully satisfy all configuration choices, they provide only a fraction of the requirements for configuration. For example, some can only guarantee 30 frames but cannot guarantee 1024p resolution. In contrast, others can ensure resolution but cannot provide a sufficient frame rate.

In recent years, most of traffic cameras can provide enough resolution and frame rate, so we try to search keywords (e.g., "highway traffic") on Youtube and download videos that meet the resolution and frame rate requirements. We select three datasets: M6, Duke, and Multi-Camera Dataset. M6 is taken from a traffic camera on the longest motorway in Britain. Duke is a video from a fixed camera placed at an intersection, and the traffic flow in the video increases or decreases periodically with the traffic light change. Since the first two datasets were sourced from a fixed camera, in order to ensure that AdaConfigure performs better in multi-camera inference, we combine three videos collected from different locations into one video dataset, Multi-Camera Dataset. The exact content of this dataset varies significantly over time and across space.

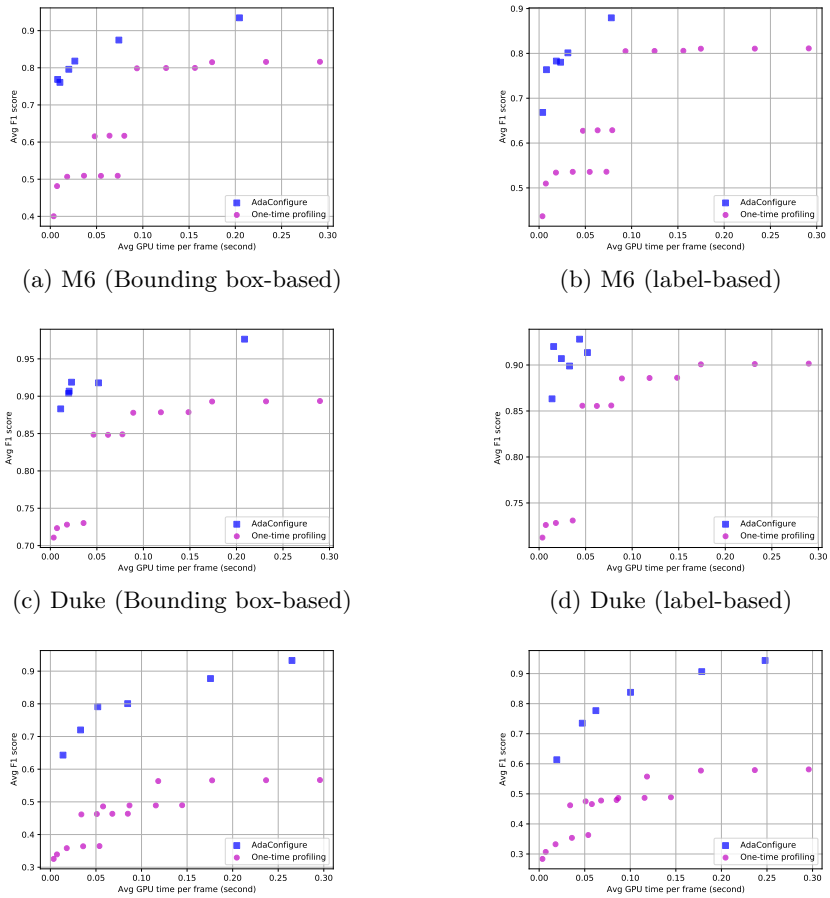
For metric, we use the F1 score as the accuracy and average GPU processing time per frame as the resource consumption because GPU is the dominant resource for most video processing workloads [10]. F1 score is the harmonic mean of precision and recall, where the precision is true positives divided by the detected objects, and the recall is true positives divided by the ground truth objects. We identify true positives using two conditions, a bounding box-based condition (only check the classified label) and a label-based condition (check the classified label and spatial overlap [1]), consistent with prior work [10, 11]. Both accuracy metrics are useful in real video analytics services and used in our experiments. Besides, to compute the accuracy of a frame that is not sampled by exact configuration, we use the location of objects from the previous sampled frame.

4.3 Experiment Parameters

In the training procedure, we build up to eight independently identical environments for each data set to speed up the training and decrease the data's dependency. For each dataset, we train the agent 5 epochs and 600 steps (5 minutes) per epoch. Some important hyperparameters in our experiments are given in Table 1.

4.4 Experiment Results

Figure 3 shows that AdaConfigure consistently outperforms the static configuration baseline (only profiling configurations once) along with resource consumption and two accuracy metrics on different datasets. Each magenta dot



(a) M6 (Bounding box-based)

(b) M6 (label-based)

(c) Duke (Bounding box-based)

(d) Duke (label-based)

(e) Multi-Camera Dataset (Bounding box-based)

(f) Multi-Camera Dataset (label-based)

Fig. 3. AdaConfigure (blue) consistently outperforms the baseline of one-time profiling (magenta) across different metrics on different datasets. Each dot represents the results of running each solution.

Notation	Value	Notation	Value
ϵ	0.8	k_1	3
γ	0.9	k_2	3
β	0.2,0.3,...,0.7	$T(train)$	1
t	1	$T(inference)$	4

Table 1. Experiment parameters.

represents one static configuration solution (one-time profiling). These static configuration solutions include some fixed expensive configurations and fixed cheap configurations. These static configuration solutions include some expen-

sive configurations, such as {FasterRCNN+InceptionResNetV2, 640p, 30fps}, {FasterRCNN+ResNet50V1, 1024p, 25fps}, and some cheap configurations, such as {SSD+ResNet152V1, 640p, 1fps}, {SSD+ResNet152V1, 640p, 5fps}. Each blue dot represents one AdaConfigure configuration solution, which is dependent on the balance factor β in reward function, presented in Section 3.4. The detailed discussion about different AdaConfigure solutions is in Section 4.4. Note that AdaConfigure's resource consumption includes both running the best configuration to get inference results and profiling cost of adaptive configuration, which detailed discussed in 4.4.

As shown in Figure 3(a)(b), AdaConfigure achieves 10-30% higher accuracy with a similar amount of resources, or achieves a similar accuracy with only 20-50% of the resources on the M6 dataset. As shown in Figure 3(c)(d), AdaConfigure achieves 10-20% higher accuracy with a similar amount of resources, or achieves a similar accuracy with only 10-20% of the resources on a Duke dataset. As shown in Figure 3(e)(f), AdaConfigure achieves 25-35% higher accuracy with a similar amount of resources, or achieves higher accuracy with only 15-25% of the resources on Multi-Camera Dataset.

In a word, AdaConfigure achieves a high accuracy of 10-35% with the same amount of computing resources, which benefits from the adaptive selection of relatively expensive configuration when the video content is complex (e.g., traffic congestion, high-speed vehicles). Also, AdaConfigure achieves a 50-90% reduction in resource consumption while achieving almost the same precision as the baseline, which benefits from the adaptive selection of relatively cheap configurations when the video content is simple (e.g., low-traffic flow).

Low Profiling Cost of Adaptive Configuration In our solution, the video chunk is passed to the AdaConfigure firstly to estimate the configuration, bringing profiling cost (extra resource consumption) to the whole system. The profiling cost, including the resource of extract k_1 embeddings and the cost of agent choosing actions. We test the average time on 30 hours video of Multi-Camera Dataset, and conclude that the average time of extract one embedding is 0.02s, and the average time of agent choosing action is 0.0006s, which can be ignored. We compute the ratio by dividing profiling time into total inference time as metric of profiling cost. The profiling cost is about 0.2-2% of the overall video analytics resource consumption. The concrete ratio depends on the concrete AdaConfigure configuration solutions, such as the solutions listed in Table 2. For instance, when using the solution of β=0.7, the profiling cost is only 0.2% of overall resource consumption on this solution; when using the solution of β=0.2, the profiling cost is 1.5%.

Different AdaConfigure Solutions to Meet Different Services In the training phase, when using different balance factors β in the reward function, we would obtain different agents (different adaptive configuration strategies). In general, if the β is bigger, we indicate that the accuracy is more important relatively, and the agent would choose a more expensive configuration. In our

balance factor β	Avg GPU time per frame (second)	Avg F1 score
0.2	0.01384	0.64307
0.3	0.03314	0.72008
0.4	0.05203	0.79096
0.5	0.08479	0.80056
0.6	0.17570	0.87731
0.7	0.26511	0.93262

Table 2. Resource consumption and bounding box-based F1 for different AdaConfigure solutions.

experiment, we set β is 0.2,0.3,...,0.7, and the results of different AdaConfigure solutions on Multi-Camera Dataset are listed in Table 2.

Table 2 shows that when the β increases, the resource consumption and the accuracy of the corresponding solution would increase, indicating the AdaConfigure solutions of big β would choose the more expensive configuration for inference. We can leverage this to train proper configuration strategy for different service demands, for example, using big β for high-accuracy demand services and using small β for low-accuracy demand services.

5 Conclusion

This paper proposes a reinforcement learning-based adaptive video analytics configuration framework, AdaConfigure. Our solution can adapt the best configuration to intrusive dynamics of video contexts, meaning that it can adaptively choose the proper configuration for the current video chunk according to the spatial and temporal features of video contexts. Therefore, our solution can adaptively update configuration strategy over time, and it adaptive to different location-camera inferences and various accuracy-demand services. In the evaluation, AdaConfigure achieves 10-35% higher accuracy with a similar amount of resources or achieves similar accuracy with only 50-90% of the resources. AdaConfigure proves to be more efficient than static solutions and only creates an overhead of 0.2-2% to the overall video analytics services.

6 Acknowledgements

This work is supported in part by NSFC (Grant No. 61872215), and Shenzhen Science and Technology Program (Grant No. RCYX20200714114523079). We would like to thank Tencent for sponsoring the research.

References

1. Everingham, M., Van Gool, L., Williams, C.K., Winn, J., Zisserman, A.: The pascal visual object classes (voc) challenge. International journal of computer vision **88**(2), 303–338 (2010)

2. FFmpeg: Ffmpeg. http://ffmpeg.org/ (2000–2018)
3. Ge, W., Yu, Y.: Borrowing treasures from the wealthy: Deep transfer learning through selective joint fine-tuning. In: CVPR. pp. 1086–1095 (2017)
4. Han, S., Shen, H., Philipose, M., Agarwal, S., Wolman, A., Krishnamurthy, A.: Mcdnn: An approximation-based execution framework for deep stream processing under resource constraints. In: MobiSys. pp. 123–136 (2016)
5. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPR. pp. 770–778 (2016)
6. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)
7. Hsieh, K., Ananthanarayanan, G., Bodik, P., Venkataraman, S., Bahl, P., Philipose, M., Gibbons, P.B., Mutlu, O.: Focus: Querying large video datasets with low latency and low cost. In: OSDI. pp. 269–286 (2018)
8. Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., Fischer, I., Wojna, Z., Song, Y., Guadarrama, S., et al.: Speed/accuracy trade-offs for modern convolutional object detectors. In: CVPR. pp. 7310–7311 (2017)
9. Hung, C.C., Ananthanarayanan, G., Bodik, P., Golubchik, L., Yu, M., Bahl, P., Philipose, M.: Videoedge: Processing camera streams using hierarchical clusters. In: SEC. pp. 115–131. IEEE (2018)
10. Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: scalable adaptation of video analytics. In: SIGCOMM. pp. 253–266 (2018)
11. Kang, D., Emmons, J., Abuzaid, F., Bailis, P., Zaharia, M.: Noscope: optimizing neural network queries over video at scale. arXiv preprint arXiv:1703.02529 (2017)
12. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., Berg, A.C.: Ssd: Single shot multibox detector. In: ECCV. pp. 21–37. Springer (2016)
13. Mao, H., Netravali, R., Alizadeh, M.: Neural adaptive video streaming with pensieve. In: SIGCOMM. pp. 197–210 (2017)
14. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602 (2013)
15. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I.: Improving language understanding by generative pre-training. URL https://s3-us-west-2. amazonaws. com/openai-assets/research-covers/languageunsupervised/language understanding paper. pdf (2018)
16. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. In: NIPS. pp. 91–99 (2015)
17. Romero, F., Li, Q., Yadwadkar, N.J., Kozyrakis, C.: Infaas: A model-less inference serving system. arXiv preprint arXiv:1905.13348 (2019)
18. Szegedy, C., Ioffe, S., Vanhoucke, V., Alemi, A.: Inception-v4, inception-resnet and the impact of residual connections on learning. arXiv preprint arXiv:1602.07261 (2016)
19. Ullah, F., Babar, M.A.: Quickadapt: Scalable adaptation for big data cyber security analytics. In: ICECCS. pp. 81–86. IEEE (2019)
20. Wang, C., Zhang, S., Chen, Y., Qian, Z., Wu, J., Xiao, M.: Joint configuration adaptation and bandwidth allocation for edge-based real-time video analytics. In: INFOCOM. pp. 1–10 (2020)
21. Zhang, H., Ananthanarayanan, G., Bodik, P., Philipose, M., Bahl, P., Freedman, M.J.: Live video analytics at scale with approximation and delay-tolerance. In: NSDI. pp. 377–392 (2017)